WAS Experiment 06

Aim: Detect and Exploit Reflected, Stored and DOM-Based XSS

Learning Objective: To demonstrate the detection and controlled exploitation of Reflected, Stored, and DOM-Based Cross-Site Scripting (XSS) vulnerabilities in a web application and understand their impact and prevention techniques.

Learning Objectives:
After completing this experiment, students will be able to:

- Understand the concept of Cross-Site Scripting (XSS).
- Explain the difference between Reflected, Stored, and DOM-Based XSS.
- Identify vulnerable input points in a web application.
- Detect reflected XSS vulnerabilities.
- Demonstrate stored XSS in a controlled laboratory environment.
- Identify DOM-based XSS vulnerabilities.
- Analyze how malicious input reaches a browser's execution context.
- Understand the security impact of XSS.
- Apply appropriate techniques to prevent XSS vulnerabilities.

Tools: Kali Linux, Terminal, Burp Suite, Web Browser-Firefox and Local or authorized vulnerable web application.

Theory:

Introduction

Cross-Site Scripting (XSS) is a web application vulnerability in which untrusted input is interpreted as executable content in a user's browser.
XSS generally occurs when an application accepts user-controlled data and places it into a web page without properly applying appropriate output encoding or other contextual protections.
An attacker may use XSS to execute unauthorized JavaScript in the security context of a vulnerable application.

Types of XSS

1. Reflected XSS

Reflected XSS occurs when malicious input is immediately returned by the web application in its response without being properly handled.
The payload is generally delivered through a request, such as a URL parameter or form input.
The general flow is:
Attacker Input
↓
Web Application
↓
Immediate Response
↓

Victim's Browser
↓
Script Execution
Reflected XSS is commonly associated with links or requests containing attacker-controlled parameters.

2. Stored XSS

Stored XSS occurs when malicious input is permanently or semi-permanently stored by the application, such as in a database.
When another user accesses the affected page, the stored content is returned and executed in the browser if appropriate protections are absent.
Stored XSS can have a broader impact because the malicious content may affect multiple users who access the affected page.

3. DOM-Based XSS

DOM-Based XSS occurs when client-side JavaScript processes attacker-controlled data and places it into a dangerous DOM context.
Unlike traditional reflected or stored XSS, the vulnerability can occur entirely within client-side JavaScript.
Examples of potentially dangerous DOM sinks include inappropriate use of:

- innerHTML
- document.write()
- eval()

Prevention Techniques
XSS vulnerabilities can be reduced by implementing appropriate security controls, including:

- Validate input where appropriate.
- Apply **context-aware output encoding**.
- Avoid inserting untrusted data into dangerous HTML/JavaScript contexts.
- Use safe DOM APIs instead of unsafe sinks.
- Implement an appropriate **Content Security Policy (CSP)**.
- Sanitize HTML when HTML input is genuinely required.
- Use secure frameworks and templating mechanisms.
- Avoid unsafe functions such as eval() where possible.
- Apply secure coding practices throughout the application lifecycle.

Procedure:

Step 1: Start Kali Linux

Start Kali Linux and launch Burp Suite.

burpsuite

Select Temporary Project and continue with the default configuration.

Expected Output:

Burp Suite starts successfully.

Step 2: Configure Firefox

Configure Firefox to use Burp Suite as the local proxy:

Proxy Address: 127.0.0.1

Proxy Port: 8080

Expected Output:

Firefox traffic is routed through Burp Suite.

Step 3: Open the Authorized XSS Laboratory

Open a local or intentionally vulnerable web application designed for XSS testing.

For example:

http://localhost/

Only use an application that is authorized for security testing.

Expected Output:

The vulnerable application's interface is displayed.

Detecting Reflected XSS

Step 4: Identify an Input Parameter

Find an input field or URL parameter that reflects user-controlled data.

For example:

http://localhost/search.php?q=test

Submit a harmless test value and observe whether it appears in the application's response.

Expected Output:

The supplied value is reflected in the response.

Step 5: Test for Reflected XSS

In the controlled laboratory application, use a basic proof-of-concept payload such as:

<script>alert('XSS')</script>

Submit the input.

Expected Output:

If the application is vulnerable, the browser executes the JavaScript and displays an alert dialog.

This demonstrates that user-controlled input has reached an executable browser context.

Detecting Stored XSS

Step 6: Identify a Persistent Input Field

Find an application feature that stores user input, such as:

- Comment field
- Message field
- Profile field
- Feedback field

Enter a harmless test value and submit it.

Expected Output:

The application stores the supplied input and displays it when the relevant page is revisited.

Step 7: Test for Stored XSS

In the controlled laboratory application, enter the proof-of-concept payload:

<script>alert('Stored XSS')</script>

Submit the input and revisit the affected page.

Expected Output:

If the application is vulnerable, the JavaScript executes when the stored content is rendered.

This demonstrates Stored XSS.

Detecting DOM-Based XSS

Step 8: Identify Client-Side Input Processing

Examine the application's client-side behavior.

Look for JavaScript that obtains data from sources such as:

- URL parameters
- URL fragments
- Browser-controlled input

and places that data into the DOM.

Potentially dangerous sinks include:

innerHTML

document.write()

eval()

Expected Output:

A client-side data flow from an attacker-controlled source to a potentially unsafe DOM sink can be identified.

Step 9: Test the DOM-Based Vulnerability

In the controlled laboratory environment, provide an appropriate test value through the identified input source.

For example, where the laboratory application specifically uses a URL fragment:

http://localhost/#test

Observe how the application processes the value.

Expected Output:

If the value reaches an unsafe DOM sink without appropriate handling, the browser may execute the supplied script.

This demonstrates DOM-Based XSS.

Step 10: Analyze Requests Using Burp Suite

Use **Burp Proxy** to capture relevant requests.

Analyze:

- HTTP method
- URL
- Parameters

- Request headers
- Response
- Reflected input

Expected Output:

The HTTP request and response associated with the XSS test can be examined.

Step 11: Compare the Three XSS Types

Compare the behavior observed during testing.

1. Reflected XSS

Input → Request → Response → Browser

2. Stored XSS

Input → Storage → Response → Browser

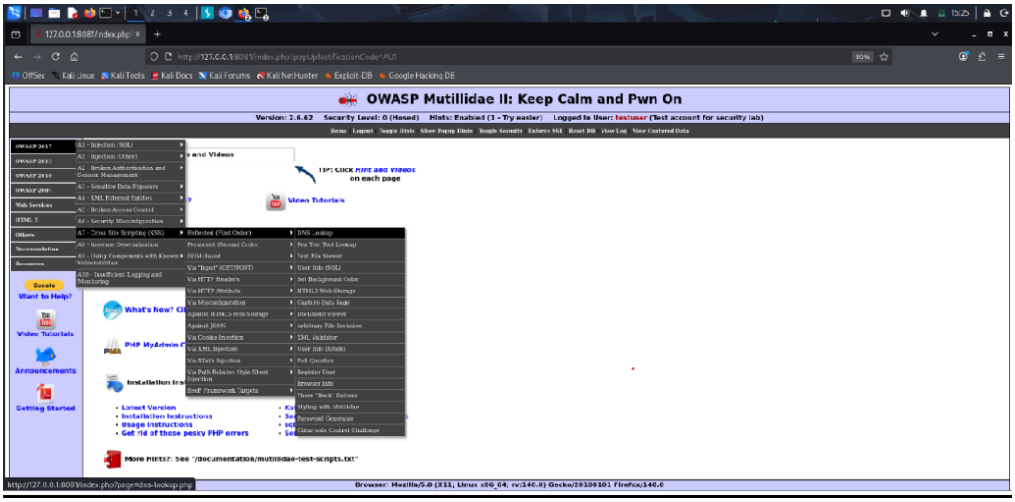
3. DOM-Based XSS

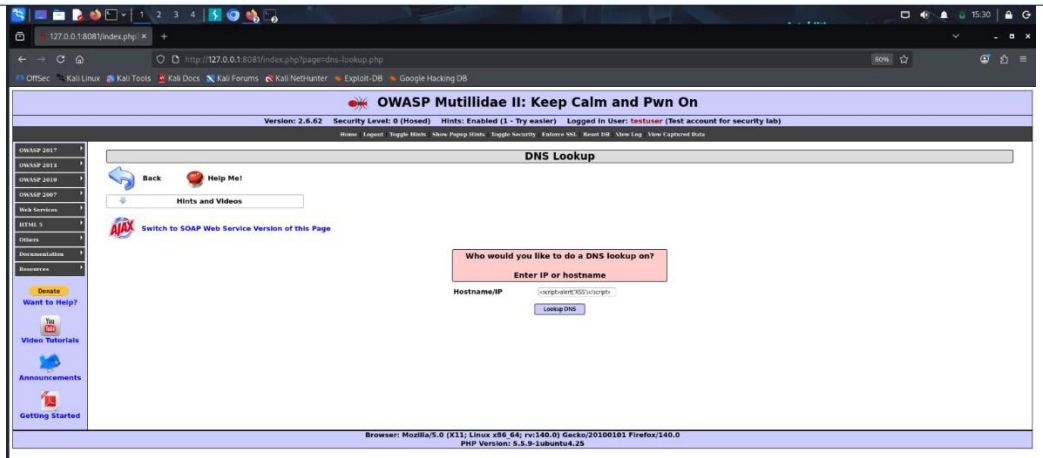
Input → Client-Side JavaScript → DOM → Browser

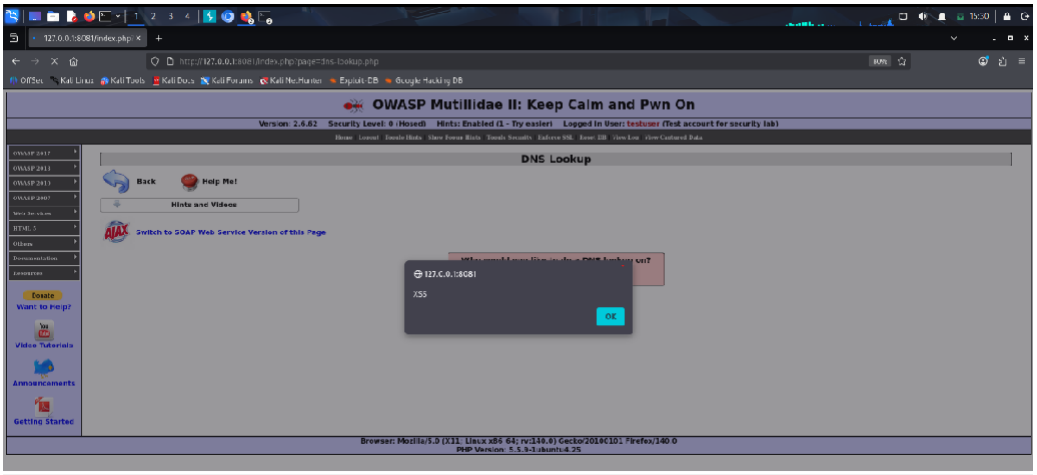
Expected Output:

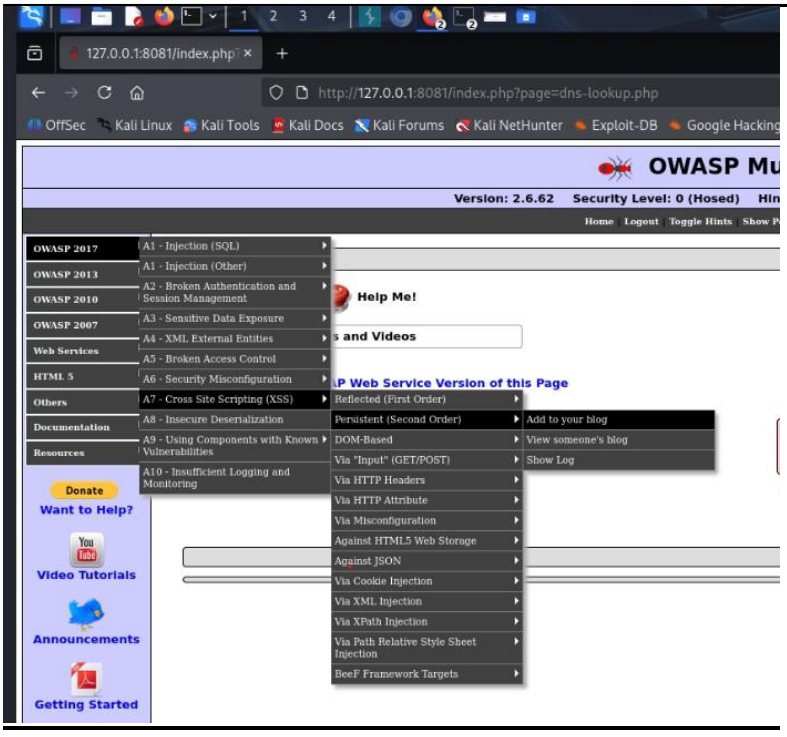
The differences between Reflected, Stored, and DOM-Based XSS are clearly identified.

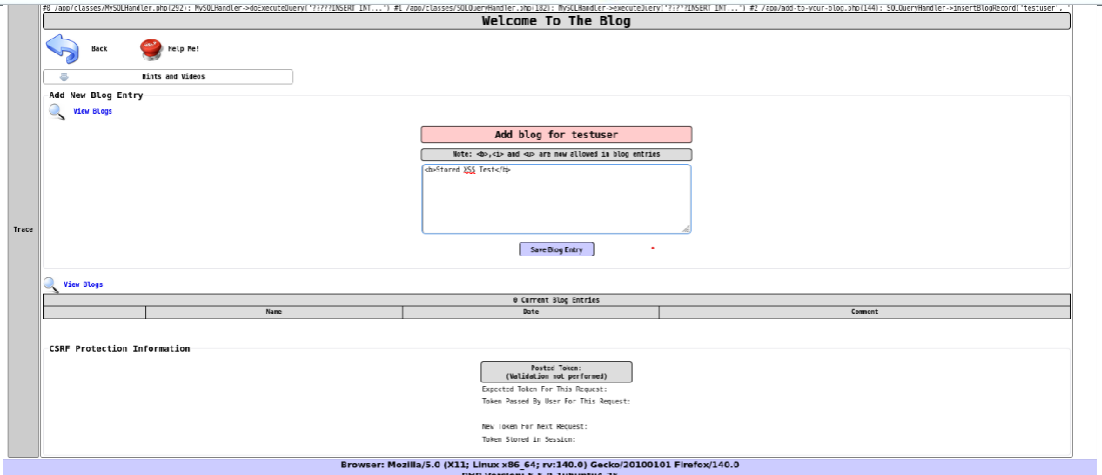
Outputs:

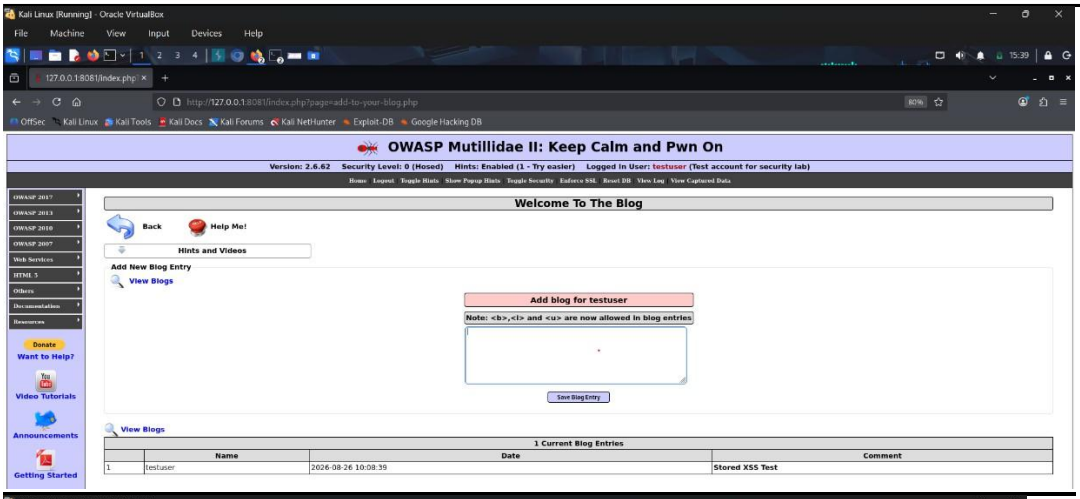


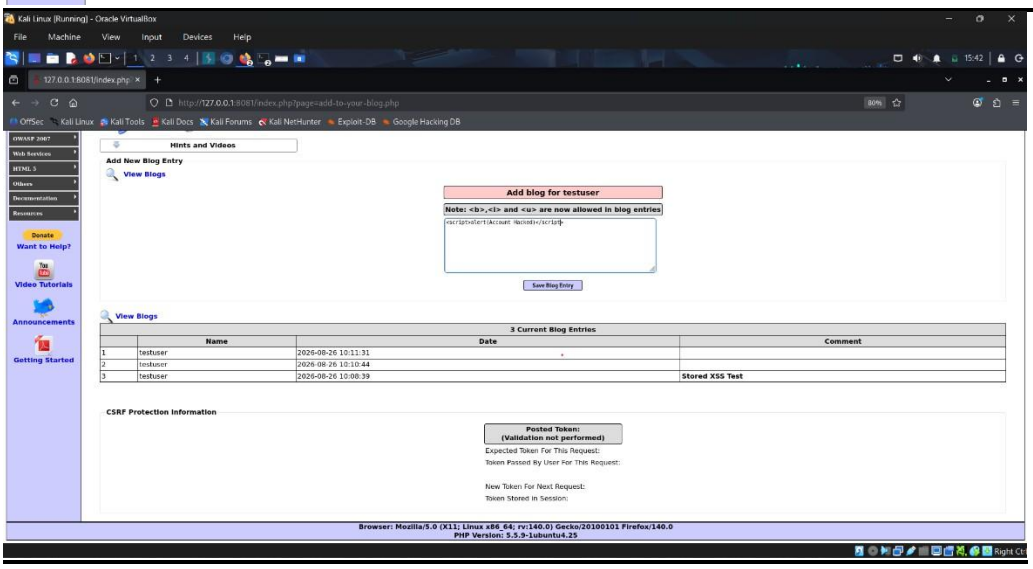

OWASP Mutillidae II: Keep Calm and Pwn On
DNS Lookup
Back
Help Me!
Hints and Videos
Switch to SOAP Web Service Version of this Page
Who would you like to do a DNS lookup on?
Enter IP or hostname
Hostname/IP
Lookup DNS

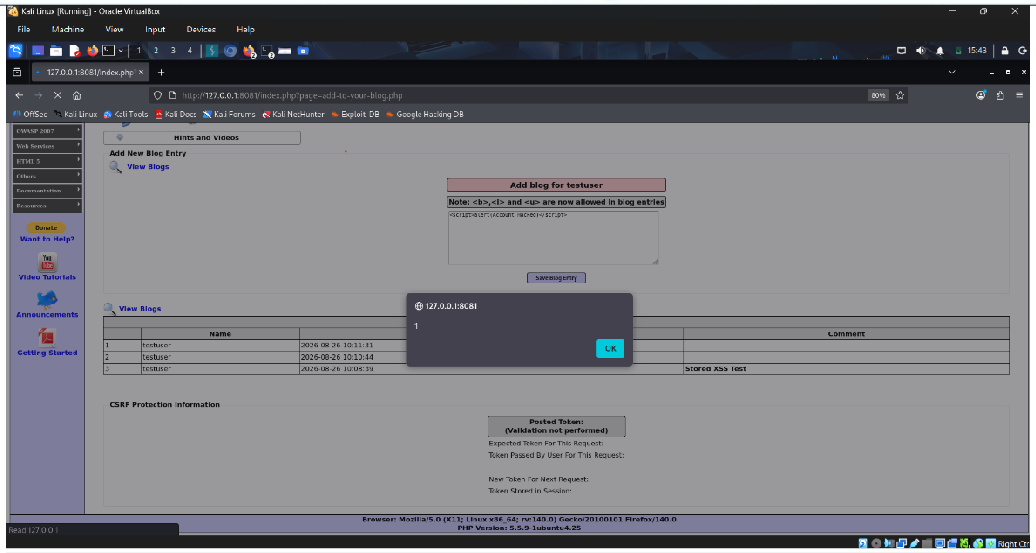

OWASP Mutillidae II: Keep Calm and Pwn On
DNS Lookup
Back
Help Me!
Switch to SOAP Web Service Version of this Page
127.0.0.1:8081
XSS
OK

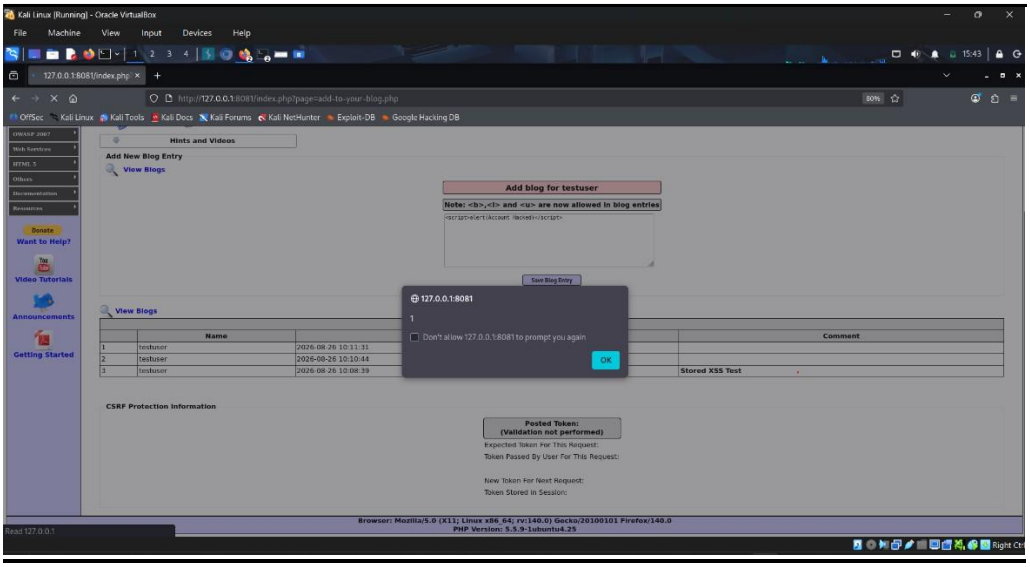

127.0.0.1:8081/index.php
http://127.0.0.1:8081/index.php?page=dns-lookup.php
OWASP 2017
OWASP 2013
OWASP 2010
OWASP 2007
Web Services
HTML 5
Others
Documentation
Resources
A1 - Injection (SQL)
A1 - Injection (Other)
A2 - Broken Authentication and Session Management
A3 - Sensitive Data Exposure
A4 - XML External Entities
A5 - Broken Access Control
A6 - Security Misconfiguration
A7 - Cross Site Scripting (XSS)
A8 - Insecure Deserialization
A9 - Using Components with Known Vulnerabilities
A10 - Insufficient Logging and Monitoring
Reflected (First Order)
Persistent (Second Order)
DOM-Based
Via "Input" (GET/POST)
Via HTTP Headers
Via HTTP Attribute
Via Misconfiguration
Against HTML5 Web Storage
Against JSON
Via Cookie Injection
Via XML Injection
Via XPath Injection
Via Path Relative Style Sheet Injection
BeeF Framework Targets
Add to your blog
View someone's blog
Show Log
Donate
Want to Help?
Video Tutorials
Announcements
Getting Started

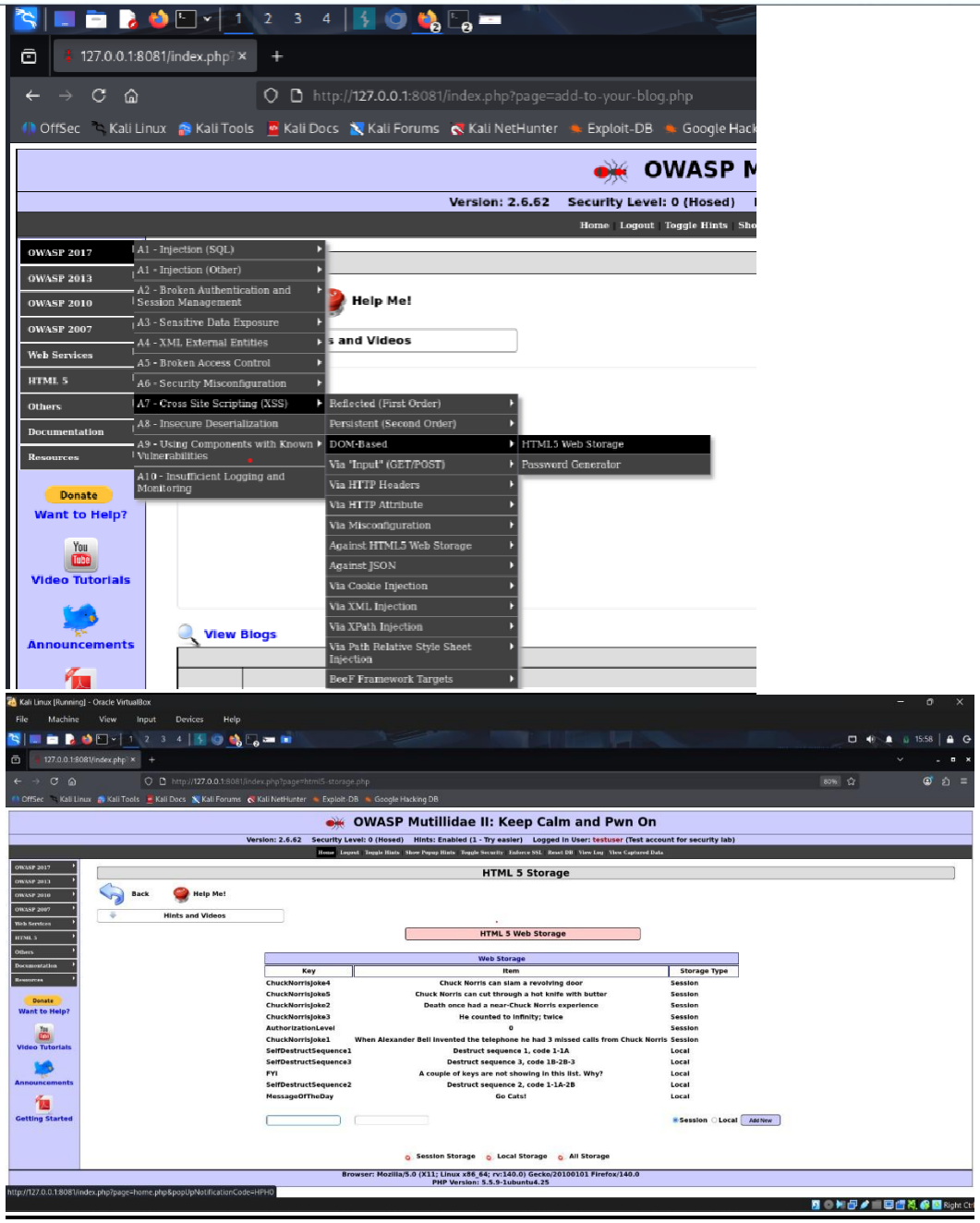

Welcome To The Blog
Back
Help Me!
Hints and Videos
Add New Blog Entry
View Blogs
Add blog for testuser
Note: ,<i> and <u> are now allowed in blog entries
Stored XSS Test
Save Blog Entry
View Blogs
0 Current Blog Entries
Name
Date
Comment
CSRF Protection Information
Posted Token:
(Validation not performed)
Expected Token For This Request:
Token Passed By User For This Request:
New Token For Next Request:
Token Stored in Session:
Browser: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
PHP Version: 5.5.9-1ubuntu4.25


Kali Linux [Running] - Oracle VirtualBox
OWASP Mutillidae II: Keep Calm and Pwn On
Version: 2.6.62 Security Level: 0 (Hosed) Hints: Enabled (1 - Try easier) Logged in User: testuser (Test account for security lab)
Welcome To The Blog
Back
Help Me!
Hints and Videos
Add New Blog Entry
View Blogs
Add blog for testuser
Note: ,<i> and <u> are now allowed in blog entries
Save Blog Entry
View Blogs
1 Current Blog Entries
Name
Date
Comment
1 testuser 2026-08-26 10:08:39 Stored XSS Test


Kali Linux [Running] - Oracle VirtualBox
Hints and Videos
Add New Blog Entry
View Blogs
Add blog for testuser
Note: ,<i> and <u> are now allowed in blog entries
Save Blog Entry
View Blogs
3 Current Blog Entries
Name
Date
Comment
1 testuser 2026-08-26 10:11:31
2 testuser 2026-08-26 10:10:44
3 testuser 2026-08-26 10:08:39 Stored XSS Test
CSRF Protection Information
Posted Token:
(Validation not performed)
Expected Token For This Request:
Token Passed By User For This Request:
New Token For Next Request:
Token Stored in Session:
Browser: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
PHP Version: 5.5.9-1ubuntu4.25


Kali Linux [Running] - Oracle VirtualBox
Add New Blog Entry
View Blogs
Add blog for testuser
Note: <b>,<i> and <u> are now allowed in blog entries
127.0.0.1:8081
1
OK
CSRF Protection Information
Posted Token:
(Validation not performed)


Kali Linux [Running] - Oracle VirtualBox
Add New Blog Entry
View Blogs
Add blog for testuser
Note: <b>,<i> and <u> are now allowed in blog entries
127.0.0.1:8081
1
Don't allow 127.0.0.1:8081 to prompt you again
OK
CSRF Protection Information
Posted Token:
(Validation not performed)



Observation:

The following observations are made:

1. Reflected XSS occurs when user-controlled input is immediately reflected into a response.
2. Stored XSS occurs when malicious input is stored by the application and later rendered.
3. DOM-Based XSS occurs through unsafe client-side processing of attacker-controlled data.
4. Burp Suite can be used to intercept and analyze HTTP requests and responses.
5. Input fields and parameters are common locations for testing XSS.
6. Improper output handling can allow browser-side script execution.
7. The three XSS types differ primarily in how attacker-controlled input reaches the browser.

Result and Discussion:

The experiment successfully demonstrated the detection and controlled exploitation of Reflected, Stored, and DOM-Based XSS vulnerabilities.

Reflected XSS was demonstrated by supplying input that was immediately reflected in the application's response. Stored XSS was demonstrated using an input that was retained by the application and executed when the affected content was displayed. DOM-Based XSS was demonstrated by examining client-side JavaScript and the flow of attacker-controlled input into the DOM.

Burp Suite assisted in analyzing the HTTP traffic and understanding how user-controlled input was processed by the application.

Conclusion:

This experiment demonstrated the three major forms of Cross-Site Scripting: Reflected XSS, Stored XSS, and DOM-Based XSS.

The experiment showed that XSS can occur when untrusted input is improperly reflected, stored and rendered, or processed unsafely by client-side JavaScript. Burp Suite was used to intercept and analyze application traffic during the testing process.

Proper input handling, context-aware output encoding, secure DOM manipulation, and security mechanisms such as CSP are important for reducing XSS risks.

All XSS testing should be performed only against intentionally vulnerable or explicitly authorized applications.

Viva Questions:

1. What is Cross-Site Scripting (XSS)?
2. What is the difference between Reflected, Stored, and DOM-Based XSS?
3. What is the main difference between Stored XSS and Reflected XSS?

For Faculty Use:

Correction Parameters	**Formative Assessment [40%]**	**Timely completion of Practical [40%]**	**Attendance / Learning Attitude [20%]**	
Marks Obtained				